arXiv:2605.06047v1 [cs.LG] 7 May 2026

TFM-Retouche: A Lightweight Input-Space Adapter for Tabular Foundation Models

Duong Nguyen **Mohammed Jawhar** **Nicolas Chesneau**
Ekimetrics

Abstract

Tabular foundation models (TFMs), such as TabPFN-2.6, TabICLv2, ConTextTab, Mitra, LimiX, and TabDPT, achieve strong zero-shot performance through in-context learning, but their inductive biases remain fixed at inference time. Adapting a pretrained TFM to a specific dataset or task typically requires either full fine-tuning, which is computationally expensive, or parameter-efficient tuning methods (PEFT) such as LoRA, which must be tailored to the internal architecture of each TFM. Furthermore, the evidence on whether weight-space fine-tuning improves accuracy or calibration is mixed [Tanna et al., 2026, Rubachev et al., 2025]. We introduce *TFM-Retouche*, a lightweight *input-space* residual adapter that is architecture-agnostic by design with respect to the frozen TFM backbone. *TFM-Retouche* learns a small residual correction in the input space to align the input data with the inductive biases of the pretrained model. The adapter is trained end-to-end through the frozen TFM, with a post-training identity guard that falls back to the unmodified TFM whenever adaptation does not help on held-out validation. On TabArena-Lite [Erickson et al., 2025] (51 datasets spanning binary classification, multiclass classification, and regression), *TabICLv2-Retouche*—the framework instantiated on TabICLv2—is the top-ranked method on the leaderboard with light per-task tuning and ensembling, lifting aggregate Elo by $+56$ over the frozen TabICLv2 base and sitting on the Pareto front of predictive quality versus both training and inference time.

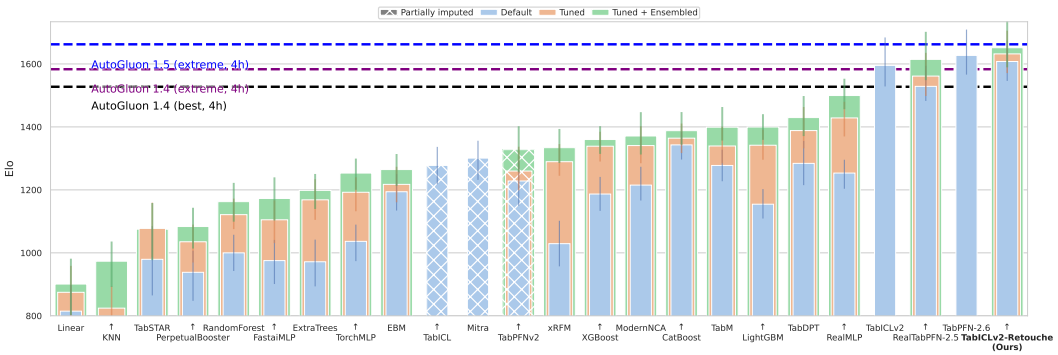


Figure 1: TabArena-Lite leaderboard. *TabICLv2-Retouche* (T+E)—our framework instantiated on frozen TabICLv2—is the top-ranked method, achieved under a $20\times$ smaller per-dataset HPO budget than every tuned peer (10 random configurations vs. TabArena's standard 200).

1 Introduction

Tabular data is central to many practical machine-learning applications, from healthcare and finance to enterprise analytics [Borisov et al., 2024, Shwartz-Ziv and Armon, 2022, Fatima and Pasha, 2017, Dastile et al., 2020]. The field was long dominated by gradient-boosted decision trees such as XGBoost [Chen and Guestrin, 2016], LightGBM [Ke et al., 2017], and CatBoost [Prokhorenkova et al., 2019], and recent deep architectures such as RealMLP [Holzmüller et al., 2025] and TabM [Gorishniy et al., 2025] have closed, and in places overtaken, the long-standing gap with well-tuned trees. Tabular foundation models (TFMs)—TabPFN [Hollmann et al., 2023], TabPFNv2 [Hollmann et al., 2025], TabPFN-2.5 [Grinsztajn et al., 2026], TabPFN-2.6 [Grinsztajn et al., 2025], Mitra [Zhang et al., 2025b], LimiX [Zhang et al., 2025a], ConTextTab [Spinaci et al., 2025], TabDPT [Ma et al., 2026], TabICL [Qu et al., 2025], and TabICLv2 [Qu et al., 2026]—have reshaped this picture: by pretraining at scale on synthetic data drawn from explicit prior generators, they encode a transferable inductive bias for tabular prediction that often rivals or surpasses tuned tree ensembles on small-to-medium problems [Erickson et al., 2025].

TFMs predict zero-shot through In-Context Learning (ICL). A labeled context is appended to the query and the prediction emerges from a single forward pass through frozen weights, with no task-specific training [Breugel and Schaar, 2024]. The conditioning step does substantial work: the in-context examples pull the prior carried by the pretrained model toward the real data of the downstream task. Yet a residual gap remains: per-task structure such as domain feature semantics, monotonicities, and dominant interactions is not always captured by what the synthetic generators emphasize, and zero-shot ICL forgoes accuracy and calibration gains that targeted adaptation can capture.

Existing approaches to that adaptation have predominantly targeted the model's weights. However, full fine-tuning is computationally expensive, and parameter-efficient fine-tuning (PEFT) methods such as LoRA [Hu et al., 2021] are lighter but tied to the backbone's internal architecture, requiring per-model choices about where and how to insert trainable modules. Beyond these practical costs, recent empirical studies have reported mixed effects on accuracy and calibration when weight-space adaptation is applied to TFMs [Rubachev et al., 2025, Tanna et al., 2026]. An interpretation is that the synthetic prior carried by the pretrained weights is fragile under further gradient updates, since the inductive bias that makes a TFM strong out-of-the-box can be eroded by even modest weight-space changes, leaving these methods with a narrow operating margin between under- and over-adaptation.

We instead adapt the *input*, not the weights. Prior TFM input-space adaptation (BETA [Liu and Ye, 2025], discussed in Section 2.2), replaces the original features with a learned encoding. That design is shaped by first-generation TabPFN's capacity constraints. With recent TFMs such as TabPFN-2.5, TabPFN-2.6, and TabICLv2 having largely lifted those capacity constraints, the bottleneck is no longer capacity but *alignment*: the frozen model's biases come from a generic synthetic prior that need not match the structure of any specific downstream task.

Our framework, *TFM-Retouche* (**T**abular **F**oundation **M**odel **Re**sidual **touch**ed **e**xtension), targets alignment by *nudging* the input rather than replacing it. A near-identity residual adapter applies a small, learned correction to the original input to align it with the inductive biases the frozen model expects, preserving feature dimensionality, treating the pretrained model as a differentiable but otherwise unmodified black box, and requiring no internal modifications. To our knowledge, this residual input-space adaptation formulation has not previously been studied for TFMs.

Our contributions are as follows:

1. The *TFM-Retouche* framework: We introduce a lightweight, configurable input-space residual adapter for frozen tabular foundation models. The adapter is dimension-preserving, architecture-agnostic with respect to the backbone TFM, and supports both classification and regression objectives. Its inner block is modular: we instantiate it as either a DCNv2 cross network [Wang et al., 2021] or a residual-bottleneck MLP block.

2. End-to-end training through a frozen TFM with post-training identity guard: We train the adapter end-to-end by backpropagating loss through the frozen TFM forward pass, leaving every pretrained weight unchanged. A per-fit identity guard scores the adapter path against the unmodified TFM and routes around the adapter at inference whenever adaptation does not improve, bounding the risk of adaptation.

3. Evaluation on TabArena-Lite: We provide a thorough evaluation *TabICLv2-Retouche*—the framework instantiated on TabICLv2—on TabArena-Lite (51 datasets spanning binary classification, multiclass classification, and regression), where it is the top-ranked method on the leaderboard with light per-task tuning and ensembling. Ablation studies are presented in Appendix E. We also include in Appendix F head-to-head comparisons against the three contemporary adaptation paradigms applicable to a frozen TabICLv2 backbone: input-space encoding via BETA [Liu and Ye, 2025] on the classification subset, LoRA-based PEFT, and full supervised fine-tuning. Appendix K extends the evaluation to the TALENT benchmark [Liu et al., 2024].

4. **Open-source release.** We will release *TFM-Retouche* as an open-source library, with different configurable settings to help practitioners adapt pretrained TFMs to their specific tasks.

2 Related Work

2.1 Tabular Foundation Models

Tabular foundation models predict zero-shot through In-Context Learning (ICL): a labeled context is appended to the query and the prediction emerges from a single forward pass through frozen weights. This paradigm was introduced by TabPFN [Hollmann et al., 2023]—a transformer pretrained on synthetic data to perform approximate Bayesian inference over a tabular prior, delivering strong zero-shot performance without task-specific gradient updates. We focus throughout on transformer-based ICL TFMs that follow this template. Alternative approaches, such as Carte [Kim et al., 2024] which pretrains a graph neural network on a real-data corpus, fall outside the scope of the present study.

Subsequent work has extended this paradigm along three axes. *Model capacity*: alternating row- and column-wise attention in TabPFNv2 [Hollmann et al., 2025], further sample/feature expansion in TabPFN-2.5 [Grinsztajn et al., 2026] and TabPFN-2.6 [Grinsztajn et al., 2025], column-then-row attention in TabICL [Qu et al., 2025] with further architectural improvements in TabICLv2 [Qu et al., 2026], and multi-scale processing with block-sparse attention in Orion-MSP [Bouadi et al., 2025]. *Pretraining prior quality*: Mitra [Zhang et al., 2025b] trains on a curated mixture of synthetic priors; TabICLv2 [Qu et al., 2026] introduces a synthetic data generation engine designed for high pretraining diversity. *Mechanism*: LimiX [Zhang et al., 2025a] extends ICL to joint-distribution modeling over masked inputs, enabling query-based conditional prediction; ConTextTab [Spinaci et al., 2025] adds specialized embeddings for different data modalities; and TabDPT [Ma et al., 2026] uses retrieval-based pretraining on real data.

Across all three axes, the open question for practitioners is alignment: a TFM trained on a generic synthetic prior is not specialized to any downstream task.

2.2 Adapting TFMs to downstream tasks

Once a TFM has been pretrained, the question becomes how to specialize its behavior to a particular downstream dataset. Existing work splits along two axes: *weight-space* adaptation, which updates the pretrained parameters of the backbone, and *input-space* adaptation, which leaves the backbone untouched and instead transforms the features that flow into it.

Weight-space adaptation (fine-tuning): Weight-space adaptation updates backbone parameters either fully or through a parameter-efficient interface such as LoRA [Hu et al., 2021]. Two recent studies provide the most relevant empirical evidence. Rubachev et al. [2025] evaluates fine-tuning strategies for TabPFNv2 on i.i.d. academic benchmarks and finds improvements over zero-shot, but reduced stability under temporal shift and on feature-rich datasets. Tanna et al. [2026] uses the TabTune library [Tanna et al., 2025] for a cross-model comparison across TALENT, OpenML-CC18, and TabZilla and finds that gains are highly model- and data-dependent: meta-learning and PEFT yield only moderate improvements under specific conditions, and full SFT often degrades accuracy and calibration. Neither study evaluates the latest TFM generation (TabICLv2 [Qu et al., 2026], TabPFN-2.5, or TabPFN-2.6 [Grinsztajn et al., 2025]). Hence, whether their conclusions generalize to this latest generation remains open.

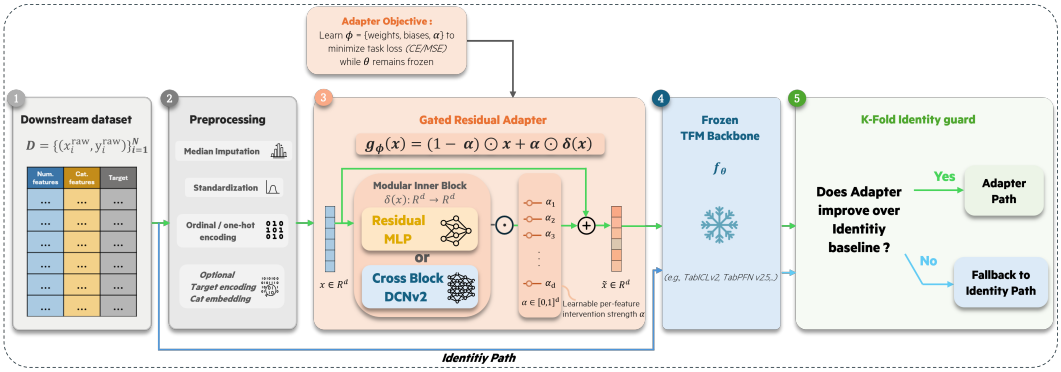


Figure 2: **Overview of *TFM-Retouche*.** A preprocessed input x is passed through a gated residual adapter $g_\phi(x) = (1-\alpha) \odot x + \alpha \odot \delta(x)$ before entering a frozen TFM. Only the adapter parameters ϕ are trained end-to-end through the frozen backbone f_θ. After training, an identity guard falls back to the unmodified TFM when adaptation does not improve held-out validation performance.

Beyond these empirical uncertainties, practical costs accrue regardless of backbone: full fine-tuning demands backpropagation and optimizer state for every backbone parameter, while PEFT methods such as LoRA are tied to the backbone's module layout, since the choice of which linear projections to wrap and at what rank must be made per architecture.

Input-space adaptation: The closest prior work is BETA [Liu and Ye, 2025]: a two-layer MLP encoder (hidden and output dimension 100, ReLU and periodic activations) trained end-to-end before a frozen TabPFN, with 16 bootstrap-sampled encoders bagged at inference. The fixed-width bottleneck reflects the narrow context capacity of first-generation TabPFN, where compressing a high-dimensional input was practically necessary. With newer TFMs such as TabICLv2, TabPFN-2.5, and TabPFN-2.6 handling much larger inputs natively, the central adaptation question shifts from capacity to alignment. *TFM-Retouche* targets alignment through a different interface and differs from BETA in three respects: (i) *dimension-preserving* rather than fixed-width bottleneck; (ii) it *nudges* the input by a small learned correction, whereas BETA *replaces* the input with a freshly learned encoding; and (iii) applicable to both classification and regression, whereas BETA is classification-only.

3 Method

3.1 Problem formulation

Let f_θ denote a pretrained tabular foundation model with frozen parameters θ, and let $D^{raw} = \{(x_i^{raw}, y_i^{raw})\}_{i=1}^N$ be a downstream dataset and $D = \{(x_i, y_i)\}_{i=1}^N$ the preprocessed data. $x_i \in \mathbb{R}^d$ is feature vector and y_i is the target. We seek a lightweight, parameter-efficient adapter $g_\phi : \mathbb{R}^d \to \mathbb{R}^d$, parameterized by ϕ with $|\phi| \ll |\theta|$ such that $f_\theta(g_\phi(X))$ outperforms $f_\theta(X^{raw})$.

Three design choices distinguish g_ϕ from prior adaptation paradigms (Section 2.2). It is (i) *dimension-preserving* (in contrast to BETA's fixed-width encoder [Liu and Ye, 2025]); (ii) *initialized near identity* ; and (iii) acts *only at the input boundary* through a differentiable forward pass of f_θ—a more general interface than weight-space PEFT methods such as LoRA, which must be tailored per backbone.

Figure 2 summarizes the framework. We describe each component in detail below.

3.2 The adapter

The core design is a gated adapter with a learnable strength parameter:

$$g_\phi(x) \;=\; (1-\alpha) \odot x \;+\; \alpha \odot \delta(x), \tag{1}$$

where α is a learnable parameter, $\delta : \mathbb{R}^d \to \mathbb{R}^d$ is a trainable transformation, and $\odot$ denotes the Hadamard product. This construction preserves the input dimensionality while allowing the adapter to apply a task-specific correction to the original feature representation. Equation 1 can equivalently

be written as a pure additive residual on x, $g_\phi(x) = x + \alpha \odot \big(\delta(x) - x\big)$, so the convex-blend and pure-residual viewpoints coincide. Appendix A gives the derivation and shows why the equivalence holds in closed form for both inner blocks of Section 3.3.

As default, we treat α as a vector $\alpha \in \mathbb{R}^d$: so that each feature dimension has its own learnable intervention strength. This lets the adapter correct some channels aggressively while leaving others close to identity. As a lower-capacity alternative, we also consider a scalar gate $\alpha \in \mathbb{R}$, that intervenes uniformly across all features. With d-fold fewer free gating parameters than the per-channel form, the scalar gate may act as a regularizer when the per-channel gate is prone to overfitting, although a controlled comparison is left to future work. Both shapes broadcast over (N, d) inputs identically and differ only in capacity.

We initialize α to a small value $\alpha_0 = 0.02$, so that $g_\phi(x) \approx x$ at the start of training and identity is recovered exactly when $\alpha = 0$ (Appendix A). The motivation is intrinsic to the problem: the frozen backbone already carries a strong default performance, so we want the TFM to see inputs essentially indistinguishable from those it learned on at the start of training, and depart from this near-identity regime to amplify or suppress the correction only as the loss signal opens specific channels. This small-initialization choice is similar in spirit to ReZero [Bachlechner et al., 2020] and LayerScale [Touvron et al., 2021].

3.3 Inner block: two variants

The inner-block transformation $\delta : \mathbb{R}^d \to \mathbb{R}^d$ is implemented as a stack of L residual layers, where the depth L is a hyperparameter. We study two variants that induce different biases while remaining interchangeable within the outer adapter of Equation 1.

Cross block (DCNv2).

$$\delta_{\text{cross}}(x) \;=\; x_L, \qquad x_{l+1} \;=\; x_0 \odot (W_l x_l + b_l) + x_l, \quad l = 0, \ldots, L-1. \tag{2}$$

where $W_l = U_l V_l^\top$ with $U_l, V_l \in \mathbb{R}^{d \times h}$, $h = \lfloor rd \rfloor$, and r is the low-rank ratio, following Wang et al. [2021]. When the inner block is low-rank, an optional activation σ may be inserted between U_l and V_l, i.e. $W_l x_l$ is replaced by $V_l\, \sigma(U_l\, x_l)$ Wang et al. [2021]; σ is a hyperparameter with the identity as default (see Appendix H). Each layer modulates the current representation x_l channel-wise by the original input x_0. Stacking L such layers yields explicit polynomial feature interactions of degree up to $L + 1$ Wang et al. [2021]. The Hadamard product with the original input x_0 imposes an *explicit* multiplicative structure and anchors each output channel to its corresponding input channel. The fitted weights are directly inspectable as pairwise feature interactions (Appendix D).

Residual MLP block.

$$\delta_{\text{MLP}}(x) \;=\; x_L, \qquad x_{l+1} \;=\; x_l + W_l^2\, \sigma\big(W_l^1 x_l + b_l^1\big) + b_l^2, \quad l = 0, \ldots, L-1, \tag{3}$$

where $W_l^1 \in \mathbb{R}^{h \times d}$, $W_l^2 \in \mathbb{R}^{d \times h}$, and the hidden width is $h = \max(h_{\min}, \lfloor rd \rfloor)$ for expansion ratio r and a minimum hidden width $h_{\min}$ (default 2). Each layer first projects the input to width h, applies activation σ (default ReLU), and projects back to d, with an optional BatchNorm at the layer output. The per-layer residual connection initializes each block close to the identity and stabilizes optimization. Structurally, this block implements a dense nonlinear transformation in which every output channel is a learned mixture of all input features, with no explicit multiplicative structure or per-channel anchoring.

The two blocks share the same outer interface, depth L, low-rank/expansion ratio r, BatchNorm option, and initialization scheme, and are matched at $O(2dh)$ trainable weights per layer at fixed r, with biases and the optional BatchNorm contributing an additional $O(d)$ parameters per layer. In the TabArena-Lite experiments (Section 4.2), we use the DCNv2 cross block as the default and treat the residual-MLP variant as a targeted ablation (Appendix E).

3.4 End-to-end training

We train the adapter parameters end-to-end while keeping the backbone TFM fully frozen. During each epoch, the training data are randomly partitioned into a context set $\mathcal{C}$ and a query set $\mathcal{Q}$, matching the in-context-learning setting of the backbone.

We optimize the adapter with AdamW [Loshchilov and Hutter, 2019], using separate parameter groups for weight matrices, biases, and the gate α. Weight decay is applied only to the matrix parameters, while biases and α are left unregularized. We further assign α a larger learning rate than the remaining parameters (a factor of 3, in our TabArena instantiation). As an alternative to using AdamW on the matrix parameters, we also consider Muon [Jordan et al., 2024], while keeping AdamW for biases and α, since Muon is defined only for two-dimensional parameters. We use cross-entropy with optional label smoothing for classification and mean squared error for regression. The default learning-rate schedule is a multi-cycle log-spaced cosine schedule [Holzmüller et al., 2025], with single-cycle cosine and constant alternatives included in the hyperparameter sweep.

3.5 Identity guard

Once training is complete, the identity guard decides, per fit, whether the trained adapter is actually used at inference or bypassed in favor of the unmodified TFM.

The guard takes an held out validation set and scores two estimators on it: the adapter path $f_\theta(g_\phi(X))$ vs. the TFM baseline that consumes the raw input $f_\theta(X^{raw})$. Scoring uses the deployment metric (for TabArena-Lite: $1 - \mathrm{AUC}$ for binary classification, log loss for multiclass, and mean squared error for regression) rather than the training loss, so that the guard's gating decision reflects inference-time performance directly. If the adapter path does not improve over the plain baseline by a small tolerance (default 0.5%), the outer estimator routes around the adapter entirely at inference and the prediction is identical to the unmodified TFM.

We report identity-fallback rates of the experiments presented in Section 4 in Appendix C.

3.6 Implementation details

The remaining implementation choices (preprocessing of numeric and categorical features and the alternative encoders considered, the input-dimension cap with its trainable low-rank projection for high-dimensional datasets, batch-size heuristic, per-epoch train/validation reshuffling, and the automatic-precision policy of the frozen TFM forward, etc.) are deferred to Appendix G. They affect efficiency and numerical conditioning but leave the adapter formulation of Equation 1 unchanged.

4 Experiments

4.1 Setup

Benchmark. We evaluate *TFM-Retouche* on TabArena-Lite [Erickson et al., 2025], following the standard TabArena protocol for data splits, model comparison, and aggregate reporting. TabArena-Lite is a curated 51-dataset covering binary classification, multiclass classification, and regression. The datasets range from 748 to 150,000 samples and from 5 to 1,777 features, providing a broad mix of small, medium, and moderately high-dimensional tabular problems. We adopt the standard AutoGluon/TabArena evaluation protocol based on 8-fold bagged cross-validation throughout. Within each fold, AutoGluon holds out a per-fold validation split that we reuse as the external validation set for the identity guard of Section 3.5.

Instantiation of *TFM-Retouche*. We denote by *TabICLv2-Retouche* the instantiation of *TFM-Retouche* using frozen TabICLv2 [Qu et al., 2026] as the backbone TFM. We choose TabICLv2 because it is one of the strongest open tabular foundation models currently available and exposes a forward pass that is convenient for our end-to-end adaptation setting. Another strong TFM is TabPFN-2.6. However, we can not use this model because of the license. The adaptation framework itself is backbone-agnostic in design, but in this paper we benchmark only the TabICLv2 instantiation. Extending the same interface to other openly available TFMs is left to future work (Section 5).

Baselines. Our primary baseline is the frozen TabICLv2 backbone without adaptation, since this isolates the contribution of the adapter from that of the underlying TFM. To place *TabICLv2-Retouche* in the broader tabular-learning landscape, we also compare against strong leaderboard baselines from TabArena-Lite, including RealTabPFN-2.5 and TabPFN-2.6 [Grinsztajn et al., 2026], the standard tabular suite comprising XGBoost [Chen and Guestrin, 2016], LightGBM [Ke et al., 2017],

CatBoost [Prokhorenkova et al., 2019], RealMLP [Holzmüller et al., 2025], TabM [Gorishniy et al., 2025], TabDPT [Ma et al., 2026], ModernNCA [Ye et al., 2025], EBM [Nori et al., 2019], as well as additional tree-based and linear baselines (full list in Appendix J). All baseline results are taken directly from the published TabArena leaderboard rather than re-run in our own environment. We omit AutoGluon from the head-to-head comparison as it is a multi-model AutoML stack rather than a single tabular model. All ranks reported below are computed within a 64-method comparison universe: the 61 external baselines from the TabArena-Lite leaderboard (the full leaderboard contains 64 methods including AutoGluon's three AutoML-stack entries, AutoGluon 1.5 (extreme, 4h), AutoGluon 1.4 (extreme, 4h), and AutoGluon 1.4 (best, 4h), which we exclude) plus the three *TabICLv2-Retouche* variants (D), (T), and (T+E). The complete 67-row leaderboard, including the AutoGluon entries, is reproduced in Appendix J.

Hyperparameter optimization. We use a lightweight hyperparameter search consisting of 11 configurations in total: 1 default configuration and 10 additional random trials. The search space covers the main adapter and optimization choices, including the number of layers, low-rank ratio, BatchNorm, learning rate, initial gate value α_0, the learning-rate multiplier for α, optimizer choice (AdamW or Muon), preprocessing variant, and learning-rate schedule. The full set of ranges and default values is listed in Appendix H. In this main sweep, the inner block is fixed to the DCNv2 cross block so that the search budget is concentrated on tuning the default instantiation of the method. The cross-versus-MLP comparison is treated separately as a focused ablation in Appendix E.

We report three evaluation protocols. **(D)** uses the default configuration without hyperparameter tuning. **(T)** selects the single best configuration for each dataset from the 11-trial search. **(T+E)** further ensembles the top configurations across the 8 validation folds.

Note that our HPO budget is deliberately small. TabArena's published leaderboard tunes each non-Retouche baseline with 200 random configurations per dataset (on top of a default), so every T or T+E peer we compare against (RealTabPFN-2.5, RealMLP, TabM, TabDPT, the GBDT suite, and others) reflects a $20\times$ larger random-search budget than *TabICLv2-Retouche* receives (10 random trials on top of a default). The *TabICLv2-Retouche* (T) and (T+E) numbers reported in Section 4.2 are therefore obtained under a substantially constrained search budget. We view them as a lower bound on what the framework can deliver, and expect a budget-matched sweep to widen the reported gains.

4.2 Main results

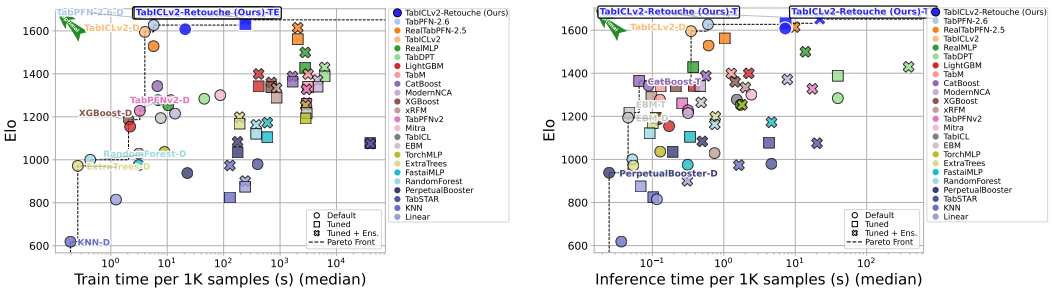
Table 1 reports the simplified TabArena-Lite leaderboard. *TabICLv2-Retouche* (T+E) reaches Elo 1651, a $+56$ gain over the unmodified TabICLv2 base (1595), and take rank 1 on the leaderboard. The single-best-config variant *TabICLv2-Retouche* (T) follows at Elo 1632 (rank 2, $+37$ vs. base). The next-best non-Retouche peers, TabPFN-2.6 (D) and RealTabPFN-2.5 (T+E), sit at Elo 1627 and 1614, respectively. *TFM-Retouche*, instantiated on TabICLv2, takes the top two slots of the leaderboard while modifying no pretrained weight and adding only 10^2–10^5 trainable parameters per dataset.

Figure 1 plots, for each method that ships both a default and a tuned variant, the Elo gain from per-dataset hyperparameter tuning. Unlike baselines whose defaults sit far below their tuned ceiling (e.g. RealMLP, LightGBM, and XGBoost each gain >170 Elo from D to T+E), *TabICLv2-Retouche* starts from a strong default at Elo 1608 that already exceeds the unmodified TabICLv2 base, and tuning further lifts it by $+43$ Elo to 1651 at rank 1. This demonstrates that *TabICLv2-Retouche* delivers a useful lift either out of the box or with a small per-task search.

Figure 3 plots Elo against per-thousand-sample training time (left) and inference time (right) for every method on the TabArena-Lite leaderboard. Appendix B shows the same data with improvability on the y-axis (Figure 4) and the corresponding critical-difference diagram (Figure 5). On the training-time axis, *TabICLv2-Retouche* (T+E) sits on the Pareto frontier as the highest-Elo operating point at 243s per 1K samples. It dominates every other (T+E) peer on both quality and training time (e.g. RealTabPFN-2.5 (T+E) at 2,060s, RealMLP (T+E) at 2,792s). TabPFN-2.6 (D) and the unmodified TabICLv2 (D) cover the fast end of the same frontier. On the inference-time axis, *TabICLv2-Retouche* (T+E) at 22s/K is the highest-Elo point on the frontier, with *TabICLv2-Retouche* (T) at 7.3s/K and the unmodified TabICLv2 / TabPFN-2.6 defaults covering the lower-Elo, faster-inference end. Here *TabICLv2-Retouche* (T+E) trades inference time against quality rather than dominating the (T+E) peers outright. The headline finding is therefore that a small input-space adapter on a frozen TFM

Table 1: Simplified TabArena-Lite leaderboard. *TabICLv2-Retouche*'s T and T+E variants use a 20× smaller per-dataset HPO budget than every tuned peer (10 random configurations vs. TabArena's standard 200). Time columns are wall-clock per 1K samples. Bold rows mark *TabICLv2-Retouche* variants. Italic row marks the unmodified TabICLv2.

Method	Elo (↑)	Norm. (↑)	Avg. rank (↓)	Train [s/K]	Pred. [s/K]
***TabICLv2-Retouche* (T+E)**	$\mathbf{1651_{-63,+85}}$	**0.665**	**10.2**	**243.17**	**22.03**
***TabICLv2-Retouche* (T)**	$\mathbf{1632_{-61,+74}}$	**0.659**	**11.0**	**243.17**	**7.33**
TabPFN-2.6 (D)	$1627_{-61,+82}$	0.643	11.3	5.75	0.60
RealTabPFN-2.5 (T+E)	$1614_{-66,+88}$	0.630	11.8	2059.94	9.79
***TabICLv2-Retouche* (D)**	$\mathbf{1608_{-62,+67}}$	**0.623**	**12.1**	**20.80**	**7.24**
TabICLv2 (D)	$1595_{-67,+89}$	*0.633*	*12.7*	*4.01*	*0.35*
RealMLP (T+E)	$1500_{-44,+54}$	0.458	17.7	2791.97	13.89
LightGBM (T+E)	$1400_{-41,+41}$	0.275	24.0	416.56	2.24
CatBoost (T+E)	$1388_{-45,+60}$	0.291	24.7	1665.53	0.56
XGBoost (T+E)	$1360_{-46,+43}$	0.231	26.7	700.96	1.44
TabPFNv2 (T+E)	$1328_{-72,+75}$	0.312	28.9	2942.08	17.37



(a) Elo vs. training time per 1K samples.

(b) Elo vs. inference time per 1K samples.

Figure 3: Quality vs. compute trade-off on TabArena-Lite: Elo vs. training time (a) and Elo vs. inference time (b). Upper-left is better; markers denote leaderboard methods. On the training-time axis (a), *TabICLv2-Retouche* (T+E) is the highest-Elo point on the Pareto frontier and dominates every other tuned-and-ensembled peer. On the inference-time axis (b), *TabICLv2-Retouche* (T+E) is the highest-Elo point on the frontier; *TabICLv2-Retouche* (T) lies on the same frontier as a lower-overhead variant.

tops the TabArena-Lite leaderboard, dominates the training-time Pareto frontier among tuned-and-ensembled methods, and remains on the inference-time frontier.

TabICLv2-Retouche's wall-clock numbers were measured on a mix of NVIDIA A10 and A100 GPUs, whereas TabArena's published times for GPU-based baselines (TabICLv2, TabPFN-2.6, RealTabPFN-2.5, TabM, TabDPT) were obtained on faster H100s. The training-time dominance and the inference-time Pareto-frontier position therefore hold *despite* a hardware handicap. Under matched hardware the time numbers would only shift further toward the fast end of the plots.

We extended the evaluation to the TALENT benchmark [Liu et al., 2024]. The result is shown in Appendix K

5 Limitations and Future Work

Several choices in this paper trade absolute performance for tractability under a constrained compute budget. First, our per-dataset random search uses only 10 configurations on top of one default, 20× smaller than TabArena's published 200-trial sweeps. Second, we share a single HPO search space across classification and regression rather than tuning per problem type, even though HPO that helps one can hurt the other [Holzmüller et al., 2025]. Problem-type-conditioned defaults or meta-learned configurations are a natural follow-up. Third, the reported wall-clock numbers were measured on NVIDIA A10 and A100 GPUs, while TabArena's published baseline times use faster H100s. Each of these factors systematically biases the reported *TabICLv2-Retouche* results downward.

We therefore view the reported leaderboard position and Pareto-frontier dominance as a lower bound on what *TabICLv2-Retouche* can deliver under matched conditions, rather than as a reflection of the framework's full potential.

We benchmark only on TabICLv2. TabPFN models (TabPFN-2.5/2.6) are not yet evaluated because of the license. *TFM-Retouche* is backbone-agnostic by design, and porting to other openly licensed TFMs is straightforward in principle but not yet evaluated.

In this paper we deliberately used a simple implementation to focus on the idea of *TFM-Retouche*: gated residual adapter with identity fallback. Several orthogonal ideas from recent tabular work fit naturally on top of this skeleton: richer per-feature numeric encodings such as PLR [Gorishniy et al., 2023] together with the robust preprocessing pipeline and better optimization tricks in RealMLP [Holzmüller et al., 2025] to give the residual branch a stronger input representation; TabM-style BatchEnsemble [Gorishniy et al., 2025] to turn a single adapter into a low-variance committee at low parameter cost; and BETA-style bagging [Liu and Ye, 2025] of the trained adapter at inference to further damp prediction variance. We expect each to compose cleanly with the framework, lifting the reported numbers above the conservative single-adapter baseline used here.

6 Conclusion

TFM-Retouche frames the adaptation of a tabular foundation model as a small, near-identity correction in *input space*: a learnable gate combined with a lightweight inner block (10^2–10^5 trainable parameters per dataset, scaling with input dimensionality), trained end-to-end through the frozen backbone, and gated at deployment by a post-training identity check on held-out validation. No pretrained weight is modified and no TFM internal-architecture choices are required, leaving the backbone a swappable knob rather than a structural commitment.

Empirically, *TabICLv2-Retouche*—the framework instantiated on TabICLv2—is the rank-1 method on TabArena-Lite at Elo 1651 (T+E, $+56$ over the frozen base), dominates the training-time Pareto frontier among tuned-and-ensembled methods, and remains on the inference-time Pareto frontier, showing that input-space adaptation can deliver state-of-the-art quality without sacrificing efficiency. These gains are obtained with only 10 random HPO trials per dataset—$20\times$ smaller than the 200-trial budget behind the published TabArena baselines.

TFM-Retouche is intended as a late-stage refinement rather than a replacement for the zero-shot TFM workflow. Modern TFMs already deliver competitive predictions with no per-dataset training, which makes them well-suited to fast iteration on the upstream decisions—data sources, feature schema, data representation, evaluation, monitoring—where most early-stage data-science effort belongs and where the frozen TFM is hard to beat as a default predictor. Once the upstream pipeline has stabilized, marginal predictive gains start to translate into real-world value, either as raw accuracy or as personalization toward a target distribution, and *TFM-Retouche* provides a small, low-friction mechanism for capturing them. *TFM-Retouche* leaves the frozen backbone untouched so the existing zero-shot pipeline remains the fallback. The per-dataset adapter (10^2–10^5 trainable parameters) trains on a single GPU at adapter-only cost rather than backbone fine-tuning cost. The identity guard (Section 3.5) routes back to the unmodified TFM on any dataset where adaptation does not improve held-out performance, making the addition strictly opt-in per dataset. And because the adapter is architecture-agnostic with respect to the backbone, the choice of TFM remains a swappable knob. This facilitate the adoption of *TFM-Retouche*, because it can incorporate future stronger TFMs naturally. Together these properties position *TFM-Retouche* as a low-risk, late-stage performance lift after the upstream data and pipeline decisions have stabilized.

Acknowledgments and Disclosure of Funding

We thank David Holzmüller for valuable discussions on deep learning and optimization techniques for tabular models, and for his advice on the choice of baselines against which to compare *TFM-Retouche*.

References

Edward I. Altman. Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4):589–609, 1968.

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huanru Henry Mao, Garrison W. Cottrell, and Julian McAuley. ReZero is All You Need: Fast Convergence at Large Depth, June 2020. URL http://arxiv.org/abs/2003.04887. arXiv:2003.04887 [cs].

Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. Deep Neural Networks and Tabular Data: A Survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6):7499–7519, June 2024. ISSN 2162-237X, 2162-2388. doi: 10.1109/TNNLS.2022.3229161. URL https://ieeexplore.ieee.org/document/9998482/.

Mohamed Bouadi, Pratinav Seth, Aditya Tanna, and Vinay Kumar Sankarapu. Orion-MSP: Multi-Scale Sparse Attention for Tabular In-Context Learning, November 2025. URL http://arxiv.org/abs/2511.02818. arXiv:2511.02818 [cs].

Boris van Breugel and Mihaela van der Schaar. Why Tabular Foundation Models Should Be a Research Priority, June 2024. URL http://arxiv.org/abs/2405.01147. arXiv:2405.01147 [cs].

Sudheer Chava and Robert A. Jarrow. Bankruptcy prediction with industry effects. *Review of Finance*, 8(4):537–569, 2004.

Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, San Francisco California USA, August 2016. ACM. ISBN 978-1-4503-4232-2. doi: 10.1145/2939672.2939785. URL https://dl.acm.org/doi/10.1145/2939672.2939785.

Xolani Dastile, Turgay Celik, and Moshe Potsane. Statistical and machine learning models in credit scoring: A systematic literature survey. *Applied Soft Computing*, 91:106263, June 2020. ISSN 1568-4946. doi: 10.1016/j.asoc.2020.106263. URL https://www.sciencedirect.com/science/article/pii/S1568494620302039.

Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Mutalik Desai, David Salinas, and Frank Hutter. TabArena: A Living Benchmark for Machine Learning on Tabular Data, November 2025. URL http://arxiv.org/abs/2506.16791. arXiv:2506.16791 [cs].

Meherwar Fatima and Maruf Pasha. Survey of Machine Learning Algorithms for Disease Diagnostic. *Journal of Intelligent Learning Systems and Applications*, 9(1):1–16, January 2017. doi: 10.4236/jilsa.2017.91001. URL https://www.scirp.org/journal/paperinformation?paperid=73781.

Yury Gorishniy, Ivan Rubachev, and Artem Babenko. On Embeddings for Numerical Features in Tabular Deep Learning, October 2023. URL http://arxiv.org/abs/2203.05556. arXiv:2203.05556 [cs].

Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling, February 2025. URL http://arxiv.org/abs/2410.24210. arXiv:2410.24210 [cs].

Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Brendan Roof, Phil Jund, Benjamin Jäger, Adrian Hayler, Dominik Safaric, Felix Jablonski Simone Alessi, Mihir Manium, Rosen Yu, Anurag Garg, Jake Robertson, Shi Bin (Liam) Hoo, Vladyslav Moroshan, Magnus Bühler, Lennart Purucker, Clara Cornu, Lilly Charlotte Wehrhahn, Alessandro Bonetto, Sauraj Gambhir, Noah Hollmann, and Frank Hutter. TabPFN-2.5, 2025.

Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, Mihir Manium, Rosen Yu, Felix Jablonski, Shi Bin Hoo, Anurag Garg, Jake Robertson, Magnus Bühler, Vladyslav Moroshan, Lennart Purucker, Clara Cornu, Lilly Charlotte Wehrhahn, Alessandro Bonetto, Bernhard Schölkopf, Sauraj Gambhir, Noah Hollmann, and Frank Hutter. TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models, February 2026. URL http://arxiv.org/abs/2511.08667. arXiv:2511.08667 [cs].

Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second, September 2023. URL http://arxiv.org/abs/2207.01848. arXiv:2207.01848 [cs].

Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, January 2025. ISSN 1476-4687. doi: 10.1038/s41586-024-08328-6. URL https://www.nature.com/articles/s41586-024-08328-6.

David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data, January 2025. URL http://arxiv.org/abs/2407.04491. arXiv:2407.04491 [cs].

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, October 2021. URL http://arxiv.org/abs/2106.09685. arXiv:2106.09685 [cs].

Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

Myung Jun Kim, Léo Grinsztajn, and Gaël Varoquaux. CARTE: Pretraining and Transfer for Tabular Learning, May 2024. URL http://arxiv.org/abs/2402.16785. arXiv:2402.16785 [cs].

Erkki K. Laitinen and Teija Laitinen. Bankruptcy prediction: Application of the Taylor's expansion in logistic regression. *International Review of Financial Analysis*, 9(4):327–349, 2000.

Si-Yang Liu and Han-Jia Ye. TabPFN Unleashed: A Scalable and Effective Solution to Tabular Classification Problems, February 2025. URL http://arxiv.org/abs/2502.02527. arXiv:2502.02527 [cs].

Si-Yang Liu, Hao-Run Cai, Qi-Le Zhou, and Han-Jia Ye. TALENT: A Tabular Analytics and Learning Toolbox, July 2024. URL http://arxiv.org/abs/2407.04057. arXiv:2407.04057 [cs].

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, January 2019. URL http://arxiv.org/abs/1711.05101. arXiv:1711.05101 [cs].

Junwei Ma, Valentin Thomas, Rasa Hosseinzadeh, Alex Labach, Hamidreza Kamkari, Jesse C. Cresswell, Keyvan Golestan, Guangwei Yu, Anthony L. Caterini, and Maksims Volkovs. TabDPT: Scaling Tabular Foundation Models on Real Data, January 2026. URL http://arxiv.org/abs/2410.18164. arXiv:2410.18164 [cs].

Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. InterpretML: A Unified Framework for Machine Learning Interpretability, September 2019. URL http://arxiv.org/abs/1909.09223. arXiv:1909.09223 [cs].

James A. Ohlson. Financial ratios and the probabilistic prediction of bankruptcy. *Journal of Accounting Research*, 18(1):109–131, 1980.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: unbiased boosting with categorical features, January 2019. URL http://arxiv.org/abs/1706.09516. arXiv:1706.09516 [cs].

Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data, May 2025. URL http://arxiv.org/abs/2502.05564. arXiv:2502.05564 [cs].

Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICLv2: A better, faster, scalable, and open tabular foundation model, February 2026. URL http://arxiv.org/abs/2602.11139. arXiv:2602.11139 [cs].

Ivan Rubachev, Akim Kotelnikov, Nikolay Kartashev, and Artem Babenko. On Finetuning Tabular Foundation Models, 2025. URL https://arxiv.org/abs/2506.08982. Version Number: 2.

Tyler Shumway. Forecasting bankruptcy more accurately: A simple hazard model. *The Journal of Business*, 74(1):101–124, 2001.

Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, May 2022. ISSN 15662535. doi: 10.1016/j.inffus.2021.11.011. URL https://linkinghub.elsevier.com/retrieve/pii/S1566253521002360.

Marco Spinaci, Marek Polewczyk, Maximilian Schambach, and Sam Thelin. ConTextTab: A Semantics-Aware Tabular In-Context Learner, November 2025. URL http://arxiv.org/abs/2506.10707. arXiv:2506.10707 [cs].

Aditya Tanna, Pratinav Seth, Mohamed Bouadi, Utsav Avaiya, and Vinay Kumar Sankarapu. TabTune: A Unified Library for Inference and Fine-Tuning Tabular Foundation Models, December 2025. URL http://arxiv.org/abs/2511.02802. arXiv:2511.02802 [cs].

Aditya Tanna, Pratinav Seth, Mohamed Bouadi, and Vinay Kumar Sankarapu. Exploring Fine-Tuning for Tabular Foundation Models, January 2026. URL http://arxiv.org/abs/2601.09654. arXiv:2601.09654 [cs].

Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with Image Transformers, April 2021. URL http://arxiv.org/abs/2103.17239. arXiv:2103.17239 [cs].

Ruoxi Wang, Rakesh Shivanna, Derek Z. Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021*, pages 1785–1797, April 2021. doi: 10.1145/3442381.3450078. URL http://arxiv.org/abs/2008.13535. arXiv:2008.13535 [cs].

Han-Jia Ye, Huai-Hong Yin, De-Chuan Zhan, and Wei-Lun Chao. Revisiting Nearest Neighbor for Tabular Data: A Deep Tabular Baseline Two Decades Later, March 2025. URL http://arxiv.org/abs/2407.03257. arXiv:2407.03257 [cs].

Xingxuan Zhang, Gang Ren, Han Yu, Hao Yuan, Hui Wang, Jiansheng Li, Jiayun Wu, Lang Mo, Li Mao, Mingchao Hao, Ningbo Dai, Renzhe Xu, Shuyang Li, Tianyang Zhang, Yue He, Yuanrui Wang, Yunjia Zhang, Zijing Xu, Dongzhe Li, Fang Gao, Hao Zou, Jiandong Liu, Jiashuo Liu, Jiawei Xu, Kaijie Cheng, Kehan Li, Linjun Zhou, Qing Li, Shaohua Fan, Xiaoyu Lin, Xinyan Han, Xuanyue Li, Yan Lu, Yuan Xue, Yuanyuan Jiang, Zimu Wang, Zhenlei Wang, and Peng Cui. LimiX: Unleashing Structured-Data Modeling Capability for Generalist Intelligence, November 2025a. URL http://arxiv.org/abs/2509.03505. arXiv:2509.03505 [cs].

Xiyuan Zhang, Danielle C. Maddix, Junming Yin, Nick Erickson, Abdul Fatir Ansari, Boran Han, Shuai Zhang, Leman Akoglu, Christos Faloutsos, Michael W. Mahoney, Cuixiong Hu, Huzefa Rangwala, George Karypis, and Bernie Wang. Mitra: Mixed Synthetic Priors for Enhancing Tabular Foundation Models, October 2025b. URL http://arxiv.org/abs/2510.21204. arXiv:2510.21204 [cs].

A Equivalent residual form of the gated adapter

Equation 1 writes the adapter as a convex blend of the raw input x and an inner-block output $\delta(x)$ under a learnable gate α. We show here that this is algebraically identical to a pure additive residual on x whose magnitude is controlled by α, and that the resulting residual has a clean closed form for both inner blocks of Section 3.3.

Distributing the gate in Equation 1 gives

$$g_\phi(x) \;=\; (1-\alpha)\odot x + \alpha \odot \delta(x) \;=\; x \;-\; \alpha \odot x \;+\; \alpha \odot \delta(x) \;=\; x \;+\; \alpha \odot \big(\delta(x) - x\big). \quad (4)$$

Defining the inner-block residual

$$r(x) \;:=\; \delta(x) - x, \quad (5)$$

the adapter is therefore a pure additive residual on x with effective gated residual $\alpha \odot r(x)$:

$$g_\phi(x) \;=\; x \;+\; \alpha \odot r(x). \quad (6)$$

Because both inner blocks defined in Section 3.3 are themselves residual stacks initialized at $x_0 = x$, the inner-block residual $r(x) = \delta(x) - x$ telescopes into a sum of per-layer increments and is available in closed form.

For the cross block (Equation 2), $x_{l+1} = x_l + x_0 \odot (W_l x_l + b_l)$, so

$$r_{\text{cross}}(x) \;=\; \delta_{\text{cross}}(x) - x \;=\; \sum_{l=0}^{L-1} x_0 \odot (W_l x_l + b_l), \quad (7)$$

which makes explicit that the cross-block contribution is a layer-wise sum of input-anchored multiplicative corrections.

For the residual-MLP block (Equation 3), $x_{l+1} = x_l + W_l^2 \sigma(W_l^1 x_l + b_l^1) + b_l^2$, so

$$r_{\text{MLP}}(x) \;=\; \delta_{\text{MLP}}(x) - x \;=\; \sum_{l=0}^{L-1} \Big(W_l^2 \sigma(W_l^1 x_l + b_l^1) + b_l^2\Big), \quad (8)$$

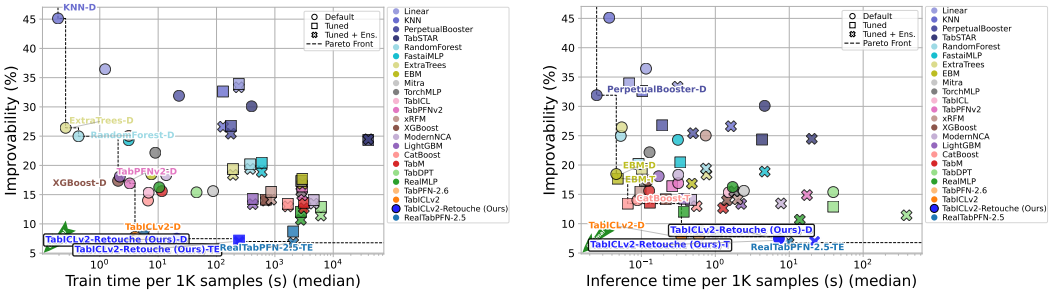
i.e. a layer-wise sum of bottlenecked nonlinear corrections. In both cases, substituting Equation 7 or Equation 8 into Equation 6 yields an explicit per-channel residual whose overall magnitude is controlled by the learned gate α.

B Additional headline-run results

This appendix collects two complementary views of the headline cross-block run reported in Section 4.2: a quality-vs-compute Pareto plot under the alternative *improvability* y-axis, and a Demšar-style critical-difference diagram that ranks the headline run against every non-AutoGluon TabArena-Lite leaderboard baseline.

Improvability vs. compute. Figure 4 replots the TabArena-Lite leaderboard data of Figure 3 with *improvability* on the y-axis (the gap to the best per-task method, averaged across tasks, as defined by Erickson et al. [2025]; lower is better). The Pareto frontier conclusion of Section 4.2 is invariant to this re-axis: *TabICLv2-Retouche* (T+E) retains its position as the highest-quality (lowest-improvability) operating point on the training-time Pareto frontier among tuned-and-ensembled methods, and remains on the inference-time frontier.

Critical-difference diagram. Figure 5 consolidates per-task ranks across TabArena-Lite into a single Demšar-style critical-difference diagram (Friedman test with Nemenyi post-hoc, $\alpha = 0.05$) over the headline *TabICLv2-Retouche* (cross) (T+E) run and every non-AutoGluon TabArena-Lite leaderboard method at its strongest protocol, for a total of 24 entries. *TabICLv2-Retouche* (T+E) achieves the lowest mean rank, ahead of TabPFN-2.6 (D), RealTabPFN-2.5 (T+E), and the unmodified TabICLv2 (D). It sits in the top significance group, which spans TabPFN-2.6 (D), RealTabPFN-2.5 (T+E), TabICLv2 (D), RealMLP (T+E), TabDPT (T+E), and TabM (T+E); within this group the pairwise rank differences are not statistically significant at $\alpha = 0.05$, so *TabICLv2-Retouche*'s top position is best read as "tied at the top" rather than as a strict separation from the strongest baselines. The remaining tabular methods (gradient-boosted trees, the legacy TFMs, and the simpler MLP and tree baselines) form a long tail of partially overlapping significance groups extending out to KNN and Linear at the bottom of the diagram.



(a) Improvability vs. training time per 1K samples. (b) Improvability vs. inference time per 1K samples.

Figure 4: Quality vs. compute trade-off on TabArena-Lite: improvability vs. training time (a) and improvability vs. inference time (b). Lower-left is better; markers denote leaderboard methods.

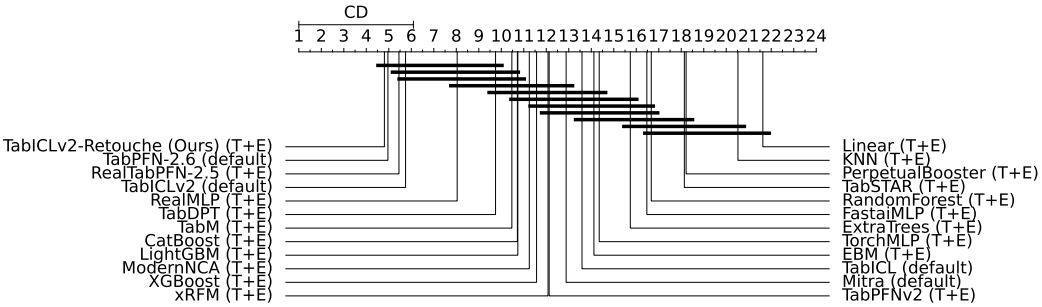


Figure 5: Critical-difference diagram (Friedman test with Nemenyi post-hoc, $\alpha = 0.05$) over per-task ranks on TabArena-Lite, covering the headline *TabICLv2-Retouche* (cross) (T+E) run and every non-AutoGluon TabArena-Lite leaderboard method at its strongest protocol (24 entries total). Mean rank is on the horizontal axis (lower is better); horizontal bars connect methods whose pairwise rank difference is not statistically significant.

C Analysis: Where does the adapter intervene?

Identity-guard activation rate. The identity guard (Section 3.5) selects the adapted path on 67.7% of runs ($3{,}040/4{,}488$, counted at the dataset $\times$ config $\times$ fold level in the official cross-block batch), where the adapter improves over the unmodified TabICLv2 base on the held-out AutoGluon bag-fold validation set; on the remaining 32.3% ($1{,}448/4{,}488$) the guard routes back to the base, ensuring the adapter is never deployed when it would not help. Because the validation set is held out from the per-fold trainer, this is a clean estimate of how often the adapter delivers a genuine gain.

Per-dataset fallback distribution. Figure 6 plots the fallback rate per-dataset. For each dataset we restrict to the single configuration that minimizes the held-out AutoGluon bag-fold validation error and report the fraction of its 8 AutoGluon bag-folds on which the guard fired. Across the 51 dataset the mean per-dataset fallback rate is 25.2% (median 25.0%, interquartile range $[12.5\%, 37.5\%]$), with $12/51$ datasets at 0% (the trained adapter wins on every fold), no dataset at 100%, and a single dataset (MIC, multiclass with 111 features) at the maximum of 75%.

The pattern is consistent with TabICLv2 being a strong starting point. Regression datasets are the most adapter-friendly (mean fallback 14.4%, median 0% across 13 datasets; $7/13$ at $\leq 5\%$), while binary classification ($n = 30$, mean 29.2%) and multiclass classification ($n = 8$, mean 28.1%) show more guard activity.

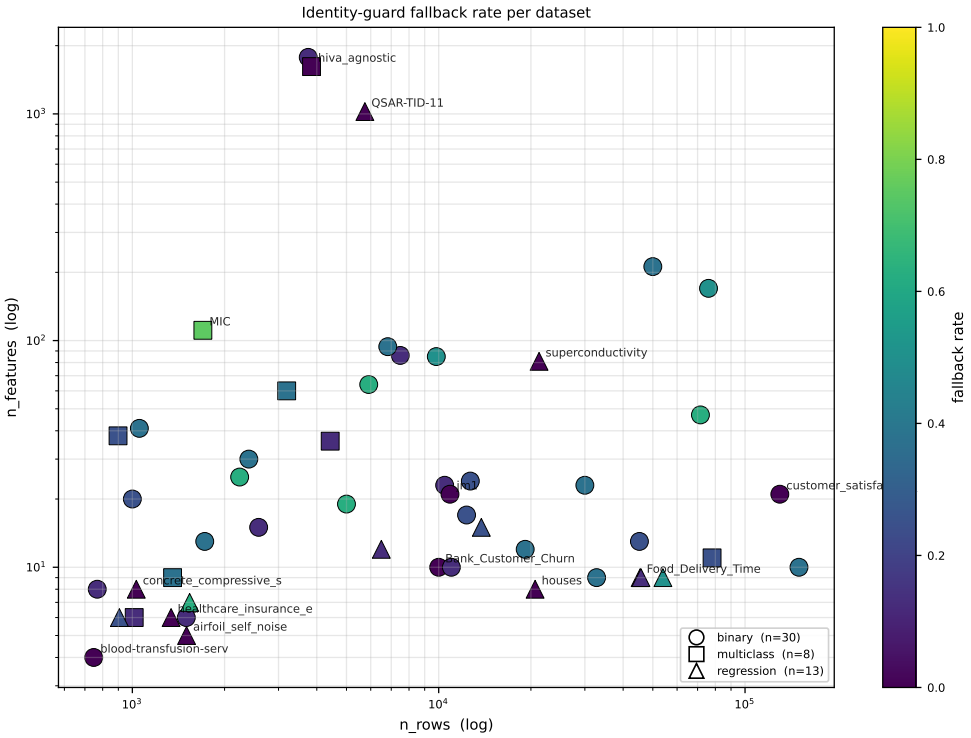


Figure 6: Per-dataset identity-guard fallback rate for the headline cross-block run of *TabICLv2-Retouche* on TabArena-Lite. Each marker is one dataset, positioned by its number of rows (horizontal, log scale) and number of features (vertical, log scale). The color encodes the fraction of the 8 AutoGluon bag-folds on which the guard routed back to the unmodified TabICLv2 base, restricted to the single configuration that minimizes the held-out AutoGluon bag-fold validation error per dataset. Marker shape encodes problem type (circle: binary, square: multiclass, triangle: regression). Datasets at the extremes (fallback rate ≥ 0.75 or ≤ 0.05) are annotated.

D Cross-block weight inspection on a financial dataset

The cross block in *TabICLv2-Retouche* computes

$$x_{l+1} = x_0 \odot (W_l\, x_l + b_l) + x_l,$$

where $\odot$ denotes the Hadamard product. Component i of the layer-$(l{+}1)$ output is a sum of terms $x_0[i] \cdot W_l[i,j] \cdot x_l[j]$, so $W_l[i,j]$ is the explicit coefficient of the multiplicative interaction $x_0[i] \cdot x_l[j]$. After fitting we can read these weights off and ask which feature pairs the adapter places weight on.

To estimate cumulative effects across L layers, we model the cross block as a vector-valued map $\mathrm{cross} : \mathbb{R}^d \to \mathbb{R}^d$, and we collapse it to the scalar function $f(x) = \sum_k \mathrm{cross}(x)_k$ by summing its d output channels with equal weight, so that the second derivative of f is a single $d \times d$ matrix. We then read off the symmetrised numerical Hessian $H[i,j] = \frac{1}{2}\big(\partial^2 f / \partial x_i\, \partial x_j + \partial^2 f / \partial x_j\, \partial x_i\big)$ via autograd, evaluated at the test-set column mean (the per-feature average of the preprocessed test inputs). Each entry $H[i,j]$ aggregates the multiplicative coupling between features i and j across all cross layers and all output channels, including BatchNorm rescaling and the inner activation between the low-rank factors $V_l\, U_l$ when one is present. We work with H rather than the analytical aggregate $\sum_l \frac{1}{2}(W_l + W_l^\top)$ because this dataset's inner block can be low-rank with a ReLU between V_l and U_l.

We analyze $|H|$ for the first bag-member of the (T) version of *TabICLv2-Retouche* (the per-dataset best-tuned configuration; see Section 4.1) on taiwanese_bankruptcy_prediction (TabArena-Lite, ID 363706, binary classification, 94 numeric financial-ratio features). Among the 15 largest off-diagonal entries of $|H|$ (Figure 7), 2 involve feature pairs whose two ratios have both appeared as

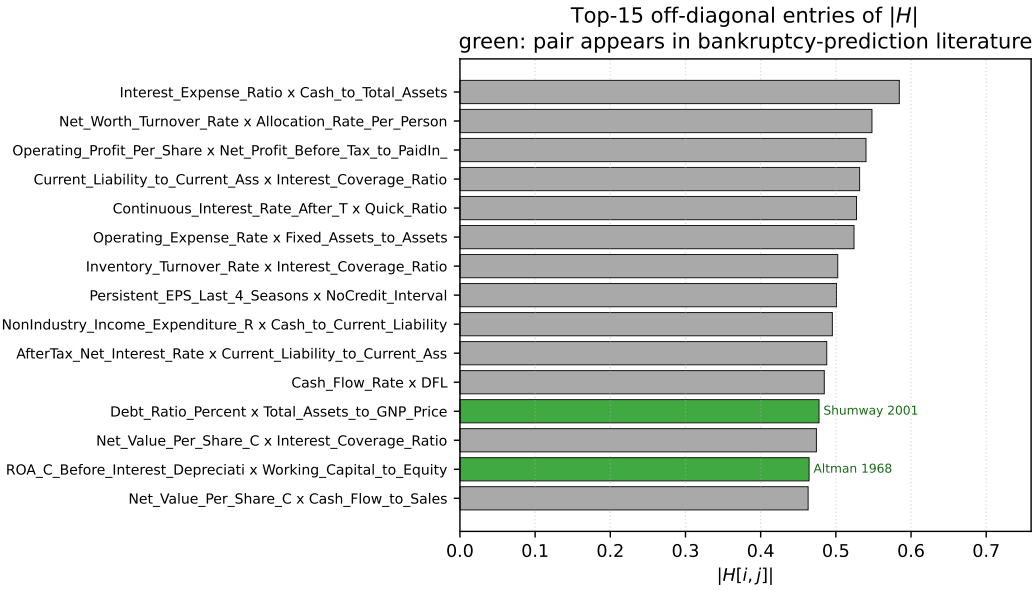


Figure 7: Magnitudes of the 15 largest off-diagonal entries of $|H|$ for the first bag-member of *TabICLv2-Retouche* (T) on taiwanese_bankruptcy_prediction (TabArena-Lite ID 363706), sorted with the largest at the top. Pairs whose two ratios both appear as separate regressors in a named bankruptcy-prediction study [Altman, 1968, Shumway, 2001, Chava and Jarrow, 2004, Ohlson, 1980] are coloured green; the remaining pairs are grey.

separate regressors in named bankruptcy-prediction studies.[1] We list those alignments below without claiming the network has identified the underlying generative process:

- Debt_Ratio_Percent $\times$ Total_Assets_to_GNP_Price. Leverage interacted with a real-deflated firm-size measure. Both ratios appear directly as regressors in the O-score logit model of Ohlson [1980]. Size and leverage also enter the discrete-time hazard models of Shumway [2001] and Chava and Jarrow [2004] as separate regressors.
- ROA_C_Before_Interest_Depreciation $\times$ Working_Capital_to_Equity. A profitability and working-capital pair within the family of variables used by the discriminant Z-score of Altman [1968].

The remaining top-15 entries involve narrower accounting variables, for example Allocation_Rate_Per_Person or Persistent_EPS_Last_4_Seasons, for which we do not have a specific reference and which we report without further interpretation.

Caveats:

- The dataset has many near-duplicate ratios (six profitability variants, five leverage variants, three liquidity variants), and the cross block carries low rank by construction. Under such multicollinearity, a low-rank fit is expected to select one proxy per economic concept rather than all of them: the textbook pair Quick_Ratio $\times$ Interest_Expense_Ratio (a liquidity-versus-debt-burden interaction) appears here with small magnitude, while the closely related Cash_to_Total_Assets $\times$ Interest_Expense_Ratio appears among the largest entries of $|H|$. The bullet list above should therefore not be read as evidence that the network has identified those specific documented variables. Substituting alternative proxies under multicollinearity is expected behavior for any low-rank fit and does not bear on which proxy is causally preferable.

[1] Explicit interaction terms have been studied in this literature: Laitinen and Laitinen [2000] add second-order and pairwise products of Cash/TA, $\mathrm{CashFlow}/TA$, and Equity/TA to a logit model and report improved accuracy 1–2 years before bankruptcy. The specific pairs probed there differ from those highlighted by the cross block here.

- This is a single illustrative dataset with named features. **We do not extrapolate** the alignments observed here to other datasets in TabArena-Lite, several of which use anonymized feature names that preclude domain interpretation. The intent of this appendix is narrow: the cross block's weights are inspectable, and on this dataset 2 of the 15 largest entries of $|H|$ correspond to feature pairs whose two ratios both appear as separate regressors in named bankruptcy-prediction studies.

E Ablations

We conducted ablation studies to understand the impact of each element in the design of *TFM-Retouche*.

Block type (cross vs. MLP vs. identity): We compare the DCNv2 cross block (our default), the residual-bottleneck MLP block, and identity (no adapter, equivalent to the frozen TabICLv2 backbone) at matched seeds and an otherwise identical configuration grid—the same 11 hyperparameter draws used in the main results (Appendix H). The cross-block run is the headline batch reported in Section 4.2; the paired MLP-block reuses every other knob unchanged.

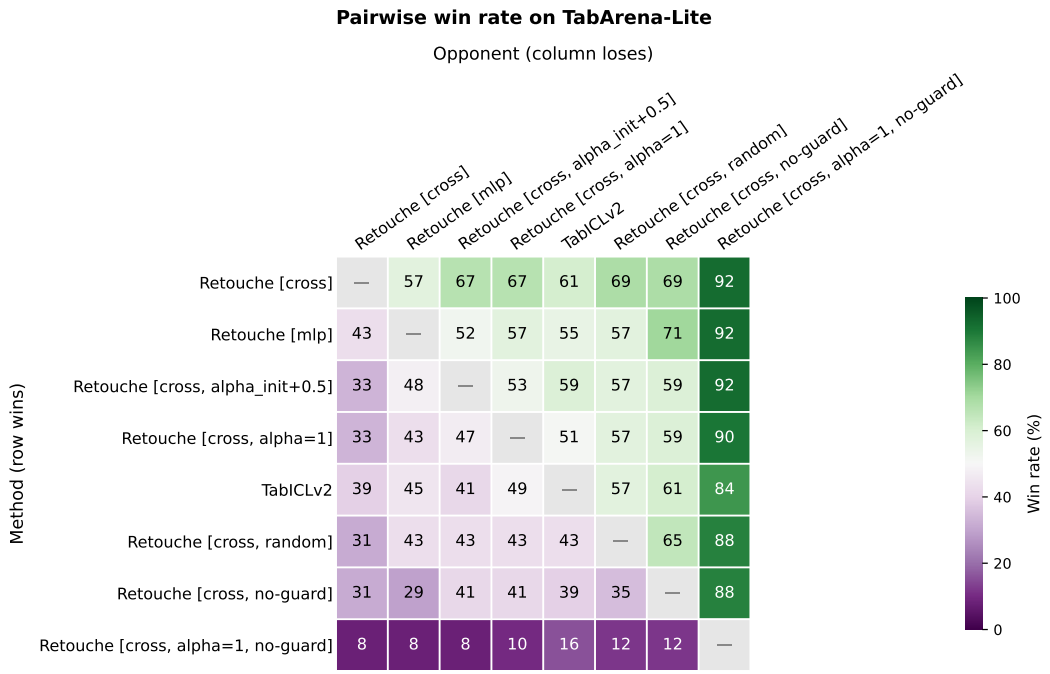
Training ablation (Random Adapter): We observed that the identity guard (Section 3.5) routed back to the unmodified frozen TabICLv2 on a sizable fraction of runs: 32.3% at the run level ($1{,}448/4{,}488$ = 51 datasets x 11 configurations x 8 folds) in the headline cross-block batch, where the trained adapter did not improve over the base on the held-out fold. This raised a natural concern: does the lift come from end-to-end training of the adapter, or could it instead be explained by randomness and the 8-fold ensembling? To isolate the contribution of training, we re-run *TabICLv2-Retouche* with the adapter randomly initialized at the beginning of the training and never updated. All the other settings are all held identical to the headline run; only the gradient updates to the adapter are removed.

Gate and identity-guard variants: The block-type and Random Adapter ablations probe two of the three safety knobs of *TabICLv2-Retouche*: the choice of inner block δ and end-to-end optimization of the adapter parameters. We complement them with four further runs that probe the trainable per-channel gate α (Section 3.2) and the identity guard (Section 3.5).

All the *TabICLv2-Retouche* variants share the headline 11-configurations and the 8-fold AutoGluon bagging protocol; each variant modifies a single knob:

- [cross] Headline cross-block run.
- [mlp] Block-type ablation: residual-bottleneck MLP δ in place of the DCNv2 cross block.
- [cross, alpha_init$+0.5$] Gate-init ablation: per-config α initialization shifted by $+0.5$ from the headline default, α remains trainable.
- [cross, alpha$=1$] Strict-freeze ablation: α initialized at 1 and held fixed; the adapter reduces to $f_\theta \circ \delta$ with no gated residual.
- TabICLv2 The unmodified frozen TabICLv2 base run at its default (D) configuration: no adapter, no gate, no identity guard. Serves as the no-adapter floor against which each *TabICLv2-Retouche* variant is measured.
- [cross, random] Training ablation (Random Adapter, above): δ randomly initialized and never updated.
- [cross, no-guard] Guard ablation: identity guard disabled, so the trained adapter is always deployed.
- [cross, alpha$=1$, no-guard] Combined ablation: α frozen at 1 and guard disabled.

We report pairwise win rates rather than Elo here: the pool consists many variants of one method, so an Elo fit would inflate the headline rating with easy wins against its own degenerate ablations. A pairwise win rate, by contrast, is a direct head-to-head statistic that does not depend on the rest of the pool. Figure 8 reports pairwise win rates (%) among the *TabICLv2-Retouche* variants (T+E) and the unmodified TabICLv2 base (D) on TabArena-Lite. Cell (i, j) is the percentage of datasets on which row method i outperforms column method j. The headline cross-block run leads the matrix, beating every other entry by at least 57–43. The cross-vs-MLP edge (57–43) is the narrowest non-degenerate gap. Both gate ablations cost the same: [cross, alpha_init$+0.5$] and [cross, alpha$=1$] each lose to



Method (row wins)	Retouche [cross]	Retouche [mlp]	Retouche [cross, alpha_init+0.5]	Retouche [cross, alpha=1]	TabICLv2	Retouche [cross, random]	Retouche [cross, no-guard]	Retouche [cross, alpha=1, no-guard]
Retouche [cross]	—	57	67	67	61	69	69	92
Retouche [mlp]	43	—	52	57	55	57	71	92
Retouche [cross, alpha_init+0.5]	33	48	—	53	59	57	59	92
Retouche [cross, alpha=1]	33	43	47	—	51	57	59	90
TabICLv2	39	45	41	49	—	57	61	84
Retouche [cross, random]	31	43	43	43	43	—	65	88
Retouche [cross, no-guard]	31	29	41	41	39	35	—	88
Retouche [cross, alpha=1, no-guard]	8	8	8	10	16	12	12	—

Figure 8: Pairwise win rates (%) among *TabICLv2-Retouche* variants (T+E) and the unmodified TabICLv2 (D) on TabArena-Lite. Cell (i, j) reports the percentage of datasets on which row method i outperforms column method j.

the headline at 33–67, indicating that the small near-identity initialization and the trainability of α contribute to the performance of the headline. The training ablation [cross, random] loses at 31–69, and the guard ablation [cross, no-guard] loses at the same 31–69. The doubly-ablated variant [cross, alpha$=1$, no-guard] wins only 8–16% across all opponents. Without either safety mechanism the trained adapter actively hurts on average.

The relative performance of the variants to the unmodified TabICLv2 base sharpens the picture. First, the headline cross-block lifts the base by 61–39 in pairwise wins and the MLP block by 55–45, so both inner blocks deliver a lift over no-adapter. Second, the strict-freeze ablation [cross, alpha$=1$] essentially *ties* the base at 51–49: with the gate frozen at 1 and the identity guard intact, the framework is no better than running TabICLv2 alone, so the small-init trainable α is what allows the lift to materialize on top of the guard.

The most striking cell is [cross, random] vs. [cross, no-guard]: the untrained-but-guarded random adapter wins 65–35 over the trained-but-unguarded adapter. On average across TabArena-Lite, the safety floor of the identity guard contributes more to the headline lift than end-to-end optimization of the adapter does.

The pairwise comparisons support a layered reading of *TabICLv2-Retouche*'s components:

- The identity guard is the single most important component: it provides more pairwise wins than training the adapter end-to-end ([cross, random] beats [cross, no-guard] 65–35), and removing it drives the trained adapter *below* the unmodified TabICLv2 base (39–61).
- The trainable gated residual is the second-most important: with the guard intact, freezing α at 1 erases the framework lift and only *ties* the base (51–49), and shifting the initialization by $+0.5$ keeps a 59–41 edge over the base while losing 33–67 to the headline default.
- Removing both safety mechanisms simultaneously is catastrophic: [cross, alpha$=1$, no-guard] wins only 8–16% across all opponents and loses to the unmodified base 16–84, validating the framework prescription that the gated residual and the identity guard are jointly load-bearing rather than redundant safeguards.

F Head-to-head: external adaptation baselines

The Ablations appendix (Appendix E) probes *TabICLv2-Retouche*'s internal design choices. Here we instead compare the framework against the three contemporary adaptation paradigms applicable to a frozen TabICLv2 stack: weight-space full supervised fine-tuning, PEFT (LoRA), and input-space encoder-and-bagging (BETA).

All four trained methods share the headline 11-configurations HPO grid (Appendix H) and the 8-fold AutoGluon bagging protocol; each baseline is run via the public reference implementation indicated below:

- TabICLv2-Retouche: The headline cross-block run reported in Section 4.2.
- TabICLv2: Unmodified frozen TabICLv2 [Qu et al., 2026] at its default configuration.
- TabICLv2-LoRA (TabTune): we vendor TabTune [Tanna et al., 2025], reusing only its `apply_tabular_lora` helper to inject LoRA adapters into the TabICLv2 stack. The surrounding training loop, optimizer setup, batching, ICL context/query split, early stopping, and identity guard are the iso-Retouche pipeline used by the headline batch (we do not invoke TabTune's own `simple_sft` or `meta-learning` fine-tuning entry points). All backbone parameters are explicitly frozen before injection so that only the LoRA `lora_A`/`lora_B` weights are trainable.
- TabICLv2-SFT (TabTune): every backbone parameter is unfrozen and updated end-to-end. The TabTune library is not invoked here; we run the iso-Retouche training loop on the unfrozen backbone with the SFT-style hyperparameters reported in TabTune's reference recipe [Tanna et al., 2025] (Adam, learning rate 10^{-5}, cross-entropy). The "(TabTune)" tag therefore identifies the recipe, not the executed code path: the per-dataset HPO grid, the bagging protocol, and the identity guard are the same as for the LoRA arm and the headline batch.
- BETA-TabICLv2: the BETA recipe [Liu and Ye, 2025] (a two-layer fixed-width MLP encoder placed before the frozen TFM and trained end-to-end) ported to the TabICLv2 backbone. The published BETA implementation targets first-generation TabPFN; we re-host its encoder/training loop on top of TabICLv2 so the head-to-head is iso-backbone with the other rows. Our wrapper currently runs single-forward inference rather than the published 16-encoder bootstrap bagging at predict time, so the BETA-TabICLv2 numbers reported here should be read as a lower bound on what the published recipe would deliver on TabICLv2.

The matrix in Figure 9 reports pairwise win rates (%) on the TabArena-Lite classification subset (Erickson et al. [2025]'s 38-task classification block, minus two tasks: Bioresponse (363620) and hiva_agnostic (363677), both excluded because the gradient-path baselines—LoRA and full SFT—could not complete on them within available GPU memory (A100); the comparison scope is therefore 36 datasets. *TabICLv2-Retouche* itself completes on all 38 classification tasks. The exclusion here is solely to keep the head-to-head matrix iso-protocol across all four trained methods.

The matrix supports four findings.

- *TabICLv2-Retouche* is the only adaptation strategy that beats the unmodified TabICLv2 base. It wins 58–42 over the base, 61–39 over both LoRA and SFT, and 92–8 over BETA. Among the five rows it is the only one with strictly above-50% win rates against every other entry in the comparison pool.
- Weight-space adaptation (both LoRA and full SFT) does not improve over zero-shot TabICLv2 on this subset. LoRA loses to the base 42–58 and full SFT loses by the same 42–58. With our matched 11-configuration HPO budget and TabArena-Lite's per-task scale, neither parameter-efficient nor full weight-space tuning yields a measurable gain over the frozen backbone, consistent with the cross-model finding of Tanna et al. [2026] that weight-space gains on already-strong zero-shot TFMs are model- and data-dependent.
- LoRA and full SFT are essentially indistinguishable on this subset (51–49 in either direction). Holding the backbone, the HPO grid, and the bagging protocol fixed, the two weight-space recipes converge to the same per-task error distribution, so the choice between PEFT and full fine-tuning is dominated by their compute cost (full SFT carries the optimizer state for every backbone parameter; LoRA does not) rather than by accuracy.

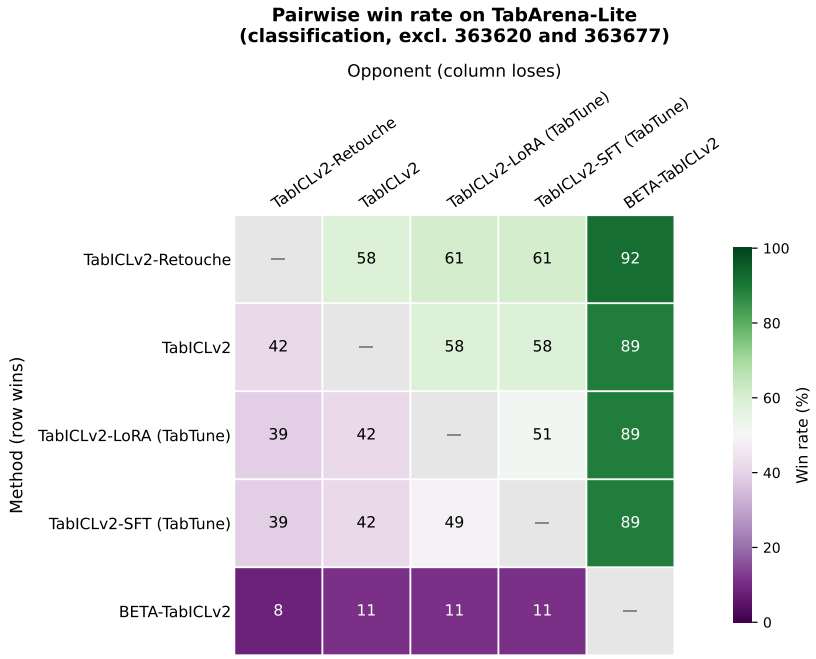


Figure 9: Pairwise win rates (%) for the four iso-backbone adaptation paradigms and the unmodified base on the TabArena-Lite classification subset (36 datasets, two datasets excluded for baseline OOM).

- Replacing the input rather than nudging it actively hurts when the backbone is already strong. BETA-TabICLv2 loses to the bare base 11–89. The BETA encoder was designed against first-generation TabPFN's capacity constraints, where compressing the input was practically necessary to keep the in-context window tractable. Mounted on TabICLv2, where that constraint no longer binds, the same fixed-width MLP encoder destroys information that the backbone could otherwise consume directly. *TabICLv2-Retouche*'s near-identity, dimension-preserving residual nudge avoids this failure mode by leaving the original feature representation intact and intervening only in proportion to a learnable per-channel gate.

Taken together, the matrix supports the central positioning of the framework: among the four adaptation paradigms compatible with a strong frozen TFM, the input-space residual nudge is the only one that delivers a measurable lift over the zero-shot backbone on TabArena-Lite classification. The bottleneck has indeed shifted from capacity (where BETA's encoder helped on TabPFN-v1) to alignment (where every weight-space path tested here ties or loses to the base, and only *TabICLv2-Retouche* clears it).

G Implementation details

Preprocessing. We map all input features to a common continuous representation before applying the adapter. Numeric features are median-imputed and standardized. Categorical features are encoded as ordinal integers and then standardized to the same scale. This choice keeps the input dimensionality fixed and ensures that all coordinates enter the adapter on a comparable scale, which is particularly important for the cross block, whose Hadamard products are sensitive to feature magnitudes.

We also consider different options for preprocessing:

- Robust encoder [Holzmüller et al., 2025] and numerical encoding, such as PLR, for numeric features [Gorishniy et al., 2023]
- Mixed preprocessing variant in which low-cardinality categorical features (at most 8 levels) are one-hot encoded, while higher-cardinality features remain ordinal-encoded and standardized [Holzmüller et al., 2025]

- Target embedding or learnable Categorical embedding for categorical features [Holzmüller et al., 2025]

These alternatives can encode numerical better, and/or expose finer-grained categorical structure, but at the cost of increasing dimensionality and weakening the "close-to-raw-input" character of the overall pipeline. Unless otherwise stated, the experiments reported in this paper use the default variant (standard scaler for numerical features and ordinal encoding for categorical features).

Other. To keep adaptation compatible with the backbone across a wide range of input dimensionalities, we cap the feature dimension seen by the TFM at 500. For datasets with $d > 500$, we therefore insert a fixed-rank trainable linear projection between the adapter and the backbone; this projection is orthogonally initialized and serves only to meet the backbone's effective input budget, not to alter the adapter formulation itself. We also support Truncated-SVD as a non-learned alternative. We additionally reshuffle the context/query split at each epoch. Finally, the frozen TFM forward follows the same automatic precision policy as TabICL: fp32 is used on smaller problems ($n < 1024$, $d < 60$), fp16 autocast on medium and large problems, and FA3 attention on sufficiently large inputs ($n \geq 10{,}240$). These choices are purely computational and leave the adapter architecture unchanged.

H Hyperparameter search space

For each TabArena-Lite dataset, we evaluate 11 configurations: one default configuration plus 10 random draws from the search space below. The same 11 configurations are evaluated on every dataset; they are sampled once with a fixed seed and reused across the headline batch and its companion batches (random adapter, no-fallback ablation, TabTune-LoRA, TabTune-SFT, and BETA head-to-heads, etc.). Numeric parameters are drawn from continuous ranges either uniformly or log-uniformly; categorical parameters are drawn uniformly from their listed sets; integer parameters are drawn uniformly over the listed inclusive range. The default values in Table 2 are those of configuration c_1 of the batch reported in Section 4.2.

I Computational details

Compute platform. All *TabICLv2-Retouche* runs reported in this paper were submitted as Databricks job runs on Microsoft Azure. Two Azure GPU SKUs were used:

- **A10:** `Standard_NV36ads_A10_v5` ($1\times$ NVIDIA A10, 24 GB HBM).
- **A100:** `Standard_NC24ads_A100_v4` ($1\times$ NVIDIA A100, 80 GB HBM).

Sharding. The headline batch reported in Section 4.2 was split across 17 Databricks job runs: 6 A10 job clusters each handling a $1/6$ shard of the lighter TabArena-Lite tasks via the official sharded suite, plus 11 A100 job clusters each pinned to a single heavier task (task IDs 363616, 363620, 363628, 363630, 363631, 363673, 363677, 363683, 363697, 363699, 363705). Each shard ran the 11 HPO configurations (one default plus 10 random draws; full search space in Appendix H) over the standard 8-fold AutoGluon bagging protocol [Erickson et al., 2025], for a total of $51 \times 11 \times 8 = 4{,}488$ end-to-end fits per batch.

Hardware-asymmetry caveat. Time-per-1K-samples figures reported on the Pareto plots (Figures 3 and 4) were measured on this A10/A100 mix, while published TabArena baselines for GPU methods were measured on faster H100s. As discussed in Section 4.2, this biases the comparison *against TabICLv2-Retouche*; under matched hardware its wall-clock numbers would only shift further toward the cheap-time end of the plots.

J Full TabArena-Lite leaderboard

For completeness, Table 3 reports the complete TabArena-Lite leaderboard for the headline cross-block run of *TabICLv2-Retouche* (Section 4.2), covering all baseline methods at all three evaluation protocols: (D) default configuration, (T) tuned over 10 random HPO configurations, and (T+E) tuned and ensembled across the 8 AutoGluon bagging folds. Methods are ranked by Elo with bootstrap

Table 2: Hyperparameter search space for *TabICLv2-Retouche*. Defaults match the first of the 11 configurations evaluated in the headline cross-block run (Section 4.2).

Hyperparameter	Distribution	Range / choices	Default
Adapter architecture			
`num_layers`	uniform choice	$\{1, 2\}$	2
`low_rank_ratio`[a]	uniform / full-rank	$[0.1, 0.5] \cup \{\text{None}\}$	0.25
`hidden_dim`	fixed	64	64
`use_batch_norm`	uniform choice	{False, True}	True
`alpha_init`	log-uniform	$[0.01, 0.1]$	0.02
`alpha_shape`	uniform choice	{per-channel, global}	per-channel
`gate_lr_factor`	log-uniform	$[2.0, 10.0]$	3.0
`block_type`[b]	fixed (cross-arm)	`cross`	cross
Optimization			
`optimizer`	uniform choice	{AdamW, Muon}	AdamW
`lr`	log-uniform	$[10^{-3},\ 1.5\times10^{-2}]$	5×10^{-3}
`weight_decay`	log-uniform	$[10^{-3},\ 5\times10^{-2}]$	3×10^{-3}
`max_grad_norm`	log-uniform	$[1.0, 5.0]$	2.0
`label_smoothing`	uniform	$[0.05, 0.30]$	0.15
`beta2`	uniform	$[0.95, 0.99]$	0.97
Training schedule			
`epochs`	integer uniform	$\{100, \ldots, 200\}$	150
`patience`	integer uniform	$\{10, \ldots, 15\}$	10
`lr_schedule`	uniform choice	{cosine, coslog4}	coslog4
Initialization and preprocessing			
`weight_init`	uniform choice	{xavier-normal, small-normal}	small-normal
`activation`	uniform choice	{None, ReLU}	None
`preprocessor`	uniform choice	{ordinal-scaled, onehot-ordinal}	ordinal-scaled

[a] `low_rank_ratio` applies only when `block_type` is `cross`; sampled as full rank (`None`) with probability $1/3$ and otherwise uniformly from $[0.1, 0.5]$. [b] The result reported in Section 4.2 has `block_type` fixed to `cross`; the paired {cross, mlp} sweep is the block-type ablation in Appendix E.

95% confidence intervals shown as $_{-l,+u}$ subscripts. Within each column, the best entry is rendered in **bold** and the second best in *italic*. Train and predict times are reported per 1,000 samples; as discussed in Appendix I, our runs use a mix of A10 and A100 GPUs while published baselines were measured on H100, which biases the time comparison *against TabICLv2-Retouche*.

Table 3: Full TabArena-Lite leaderboard for the headline cross-block run of *TabICLv2-Retouche* (Section 4.2). Ranked by Elo. Best per column in **bold**, second in *italic*. Train/predict times are per 1,000 samples.

Model	Elo (↑)	Norm. score (↑)	Avg. rank (↓)	Harm. mean rank (↓)	#wins (↑)	Improvability (↓)	Train time per 1K [s]	Predict time per 1K [s]
AutoGluon 1.5 (extreme, 4h)	**1662**$_{-84,+105}$	**0.710**	**9.8**	**3.2**	**9.0**	**5.1%**	293.65	4.36
TabICLv2-Retouche (Ours) (T+E)	*1651*$_{-63,+85}$	*0.665*	*10.2*	5.0	1.3	7.0%	243.17	22.03
TabICLv2-Retouche (Ours) (T)	1632$_{-61,+74}$	0.659	11.0	4.3	5.3	7.1%	243.17	7.33
TabPFN-2.6 (D)	1627$_{-61,+82}$	0.643	11.3	5.3	3.0	7.8%	5.75	0.60
RealTabPFN-2.5 (T+E)	1614$_{-66,+88}$	0.630	11.8	4.2	4.0	*6.8%*	2059.94	9.79
TabICLv2-Retouche (Ours) (D)	1608$_{-62,+67}$	0.623	12.1	5.9	1.3	7.5%	20.80	7.24
TabICLv2 (D)	1595$_{-67,+89}$	0.633	12.7	*4.0*	*6.0*	7.8%	4.01	0.35
AutoGluon 1.4 (extreme, 4h)	1583$_{-68,+82}$	0.605	13.3	5.8	2.0	8.2%	556.15	6.31
RealTabPFN-2.5 (T)	1561$_{-64,+74}$	0.556	14.3	6.9	2.0	8.7%	2059.94	1.03
RealTabPFN-2.5 (D)	1529$_{-47,+66}$	0.508	16.0	9.0	0.0	8.9%	5.71	0.61
AutoGluon 1.4 (best, 4h)	1528$_{-59,+56}$	0.502	16.1	7.7	1.0	9.7%	1754.94	1.77
RealMLP (T+E)	1500$_{-44,+54}$	0.458	17.7	10.6	1.0	10.7%	2791.97	13.89
TabDPT (T+E)	1430$_{-59,+69}$	0.403	22.0	7.4	3.0	11.4%	6154.73	386.17
RealMLP (T)	1428$_{-59,+52}$	0.378	22.0	12.8	0.0	12.0%	2791.97	0.37
LightGBM (T+E)	1400$_{-41,+41}$	0.275	24.0	18.9	0.0	13.4%	416.56	2.24

(continues on next page)

(continued)

Model	Elo (↑)	Norm. score (↑)	Avg. rank (↓)	Harm. mean rank (↓)	#wins (↑)	Improva-bility (↓)	Train time per 1K [s]	Predict time per 1K [s]
TabM (T+E)	$1399_{-43,+64}$	0.333	24.0	15.4	0.0	12.7%	3133.91	1.27
TabDPT (T)	$1388_{-57,+75}$	0.355	24.7	10.4	0.0	12.9%	6154.73	39.45
CatBoost (T+E)	$1388_{-45,+60}$	0.291	24.7	16.9	0.0	13.0%	1665.53	0.56
ModernNCA (T+E)	$1371_{-59,+76}$	0.343	25.9	13.1	1.0	13.5%	4618.50	7.74
CatBoost (T)	$1364_{-48,+47}$	0.265	26.4	17.6	0.0	13.4%	1665.53	0.07
XGBoost (T+E)	$1360_{-46,+43}$	0.231	26.7	19.2	0.0	14.0%	700.96	1.44
CatBoost (D)	$1343_{-47,+42}$	0.229	27.8	18.5	0.0	14.0%	6.70	0.09
LightGBM (T)	$1342_{-47,+50}$	0.221	27.9	22.9	0.0	14.3%	416.56	0.38
ModernNCA (T)	$1341_{-57,+62}$	0.263	28.0	14.7	1.0	14.1%	4618.50	0.47
TabM (T)	$1339_{-56,+62}$	0.261	28.1	17.9	0.0	13.6%	3133.91	0.13
XGBoost (T)	$1339_{-49,+46}$	0.207	28.1	20.0	0.0	14.1%	700.96	0.21
xRFM (T+E)	$1334_{-49,+60}$	0.252	28.5	18.0	0.0	14.1%	866.11	2.01
TabPFNv2 (T+E)	$1328_{-72,+75}$	0.312	28.9	11.4	1.0	14.9%	2942.08	17.37
Mitra (D)	$1301_{-70,+56}$	0.252	30.8	14.5	1.0	15.6%	87.34	2.43
xRFM (T)	$1290_{-45,+56}$	0.184	31.6	16.5	1.0	15.5%	866.11	0.10
TabDPT (D)	$1284_{-70,+72}$	0.253	32.0	16.4	0.0	15.4%	45.42	39.41
TabM (D)	$1278_{-51,+50}$	0.199	32.5	24.1	0.0	15.6%	11.56	0.13
TabICL (D)	$1278_{-60,+59}$	0.223	32.5	13.4	1.0	15.3%	6.86	1.52
EBM (T+E)	$1264_{-52,+50}$	0.156	33.4	24.3	0.0	16.8%	2961.52	0.48
TabPFNv2 (T)	$1260_{-62,+78}$	0.214	33.8	19.8	0.0	16.4%	2942.08	0.26
RealMLP (D)	$1253_{-50,+43}$	0.116	34.2	26.2	0.0	16.2%	10.44	*1.71*
TorchMLP (T+E)	$1253_{-54,+46}$	0.138	34.2	28.6	0.0	15.5%	2832.80	1.80
TabPFNv2 (D)	$1228_{-79,+66}$	0.189	36.0	18.6	0.0	16.9%	3.27	0.32
EBM (T)	$1218_{-57,+56}$	0.114	36.8	26.4	0.0	17.7%	2961.52	**0.05**
ModernNCA (D)	$1215_{-49,+59}$	0.114	37.0	18.5	1.0	18.3%	13.74	0.32
ExtraTrees (T+E)	$1198_{-60,+53}$	0.100	38.2	28.5	0.0	18.4%	191.44	0.76
EBM (D)	$1194_{-60,+57}$	0.111	38.5	20.1	1.0	18.5%	7.66	**0.05**
TorchMLP (T)	$1192_{-61,+48}$	0.102	38.6	30.9	0.0	17.3%	2832.80	0.11
XGBoost (D)	$1187_{-54,+55}$	0.098	39.0	20.6	1.0	17.4%	*2.06*	0.12
FastaiMLP (T+E)	$1173_{-71,+68}$	0.096	40.0	30.4	0.0	18.9%	594.95	4.65
ExtraTrees (T)	$1168_{-64,+66}$	0.101	40.3	26.5	0.0	19.4%	191.44	0.10
RandomForest (T+E)	$1163_{-64,+60}$	0.082	40.7	32.0	0.0	19.4%	377.08	0.75
LightGBM (D)	$1154_{-46,+48}$	0.071	41.2	36.2	0.0	18.1%	2.20	0.17
RandomForest (T)	$1121_{-46,+51}$	0.047	43.5	37.0	0.0	20.2%	377.08	0.09
FastaiMLP (T)	$1105_{-74,+62}$	0.060	44.5	32.5	0.0	20.5%	594.95	0.34
PerpetualBooster (T+E)	$1084_{-70,+60}$	0.066	45.9	37.6	0.0	25.4%	176.26	0.50
TabSTAR (T)	$1078_{-92,+82}$	0.109	46.3	18.7	0.0	24.4%	39913.02	4.29
TabSTAR (T+E)	$1075_{-99,+83}$	0.108	46.5	16.4	1.0	24.5%	39913.02	20.17
TorchMLP (D)	$1036_{-63,+54}$	0.023	48.8	44.9	0.0	22.2%	8.96	0.13
PerpetualBooster (T)	$1035_{-66,+55}$	0.035	48.9	44.6	0.0	26.8%	176.26	0.19
xRFM (D)	$1029_{-73,+73}$	0.049	49.3	39.2	0.0	25.0%	3.14	0.74
RandomForest (D)	$1000_{-58,+58}$	0.013	50.9	44.5	0.0	25.0%	*0.43*	**0.05**
TabSTAR (D)	$980_{-115,+106}$	0.081	52.0	19.5	1.0	30.1%	398.77	4.65
FastaiMLP (D)	$975_{-75,+66}$	0.019	52.2	48.8	0.0	24.3%	3.12	0.31
KNN (T+E)	$974_{-82,+63}$	0.024	52.3	46.5	0.0	26.6%	129.10	1.63
ExtraTrees (D)	$972_{-79,+71}$	0.014	52.4	48.5	0.0	26.4%	0.26	**0.05**
PerpetualBooster (D)	$938_{-91,+69}$	0.027	54.1	42.9	0.0	31.9%	22.75	0.02
Linear (T+E)	$901_{-98,+81}$	0.023	55.8	26.6	1.0	33.4%	240.73	0.31
Linear (T)	$875_{-114,+84}$	0.017	56.9	36.7	0.0	33.9%	240.73	0.07
KNN (T)	$824_{-92,+70}$	0.013	58.8	56.1	0.0	32.6%	129.10	0.10
Linear (D)	$815_{-106,+84}$	0.005	59.2	56.4	0.0	36.4%	1.23	0.12
KNN (D)	$619_{-109,+74}$	0.000	64.1	63.7	0.0	45.1%	**0.19**	**0.04**

K Evaluation on the TALENT benchmark

We run *TabICLv2-Retouche* on the classification subset of the TALENT benchmark of Liu et al. [2024] restricted to datasets with at most 10 classes (170 binary and multiclass classification datasets). For this experiment **we use only the default configuration** for *TabICLv2-Retouche*.

Setup: We run three random training seeds per dataset and reuse the preprocessing pipeline of the TabArena-Lite headline run (Section 4.2). The main baseline is the unmodified TabICLv2 default,

Table 4: Mean rank on the TALENT classification subset (Accuracy, 170 datasets with at most 10 classes). Methods shipped with the TALENT toolbox are re-ranked over this dataset universe; their own coverage may be a subset, in which case the column n reflects that subset. Best per column in **bold**.

Metric	Method	Mean rank (↓)	n
Accuracy	***TabICLv2-Retouche***	3.83	170
Accuracy	TabICLv2	4.05	170
Accuracy	TabR	9.32	161
Accuracy	RealMLP	9.82	161
Accuracy	ModernNCA	10.10	159
Accuracy	CatBoost	10.42	161
Accuracy	LightGBM	10.78	161
Accuracy	XGBoost	11.97	161
Accuracy	FTT	12.88	161
Accuracy	MLP_PRL	13.30	161
Accuracy	Resnet	14.94	161

also at three seeds. The remaining baselines are the per-method, per-task numbers shipped with TALENT [Liu et al., 2024]; we re-rank them in our pool rather than re-running them.

TabICLv2-Retouche completes all three seeds on every one of the 170 in-scope datasets. Ten of those datasets, in the upper tail of sample size or feature count, required a tighter context budget than the headline default in order to fit the differentiable forward pass through TabICLv2 within A100 memory and the CUDA kernel-grid limit. We cap the per-epoch in-context window at `max_samples` $= 15{,}000$ to `60,000` rows depending on the dataset (chosen by halving from $60{,}000$ until the forward fits), and on the two largest datasets we additionally chunk the query side at `max_query` $= 8{,}192$ to keep `train+query` tokens below the $65{,}535$ kernel-grid limit. The unmodified TabICLv2 baseline reaches comparable in-context budgets via its stock sklearn predict pipeline, which subsamples internally before each forward [Qu et al., 2026], so the deviation is approximately iso-context with the baseline rather than an asymmetric advantage.

Result: Table 4 reports the top mean per-task rank on Accuracy over the 170 in-scope datasets. *TabICLv2-Retouche* is the top-ranked method on the leaderboard at mean rank 3.83, ahead of the TabICLv2 base (4.05) and well clear of the next-best non-TabICL method TabR (9.32, 161 datasets); every other entry on the leaderboard sits at mean rank ≥ 9.8.

Table 5: Paired wins / ties / losses for *TabICLv2-Retouche* vs. the TabICLv2 default on the TALENT classification subset restricted to datasets with at most 10 classes (170 datasets per metric). "Win" means *TabICLv2-Retouche* strictly improves the seed-averaged metric over the base on that dataset.

Metric	Wins	Ties	Losses
Accuracy	84	14	72
AUC	97	4	69
Avg. Precision	84	11	75
Avg. Recall	85	12	73
F1	81	11	78
LogLoss	109	0	61
Total	**540**	**52**	**428**

Head-to-head against the backbone base (*TabICLv2-Retouche* vs. TabICLv2): Table 5 reports per-metric paired wins, ties, and losses across the 170 datasets. *TabICLv2-Retouche* records more wins than losses on every metric. The aggregate over the $1{,}020$ paired cells is 540 wins, 52 ties, and 428 losses. Restricted to the cells where the two methods produce different seed-averaged scores (i.e. Ties are excluded), the per-metric win rate is 53.8% on Accuracy, 58.4% on AUC, 52.8% on Avg. Precision, 53.8% on Avg. Recall, 50.9% on F1, and 64.1% on LogLoss. The F1 margin is the smallest (81 wins vs. 78 losses, a 3-dataset edge). The gap on the remaining metrics is more substantial, with LogLoss the most decisive.